

Approach & Design Document

Secure Face Registration and Verification Framework using Face Quality Assessment and Liveness Detection

The Problem

Think of this project as building the brain of a security guard for face-based login. The guard has two jobs. First, when someone signs up (registration), take a clear photo of their face and remember it. Second, when someone tries to log in later (verification), look at their face again and answer two questions: is this a real, live person in front of the camera right now, and does this face actually belong to the person who signed up.

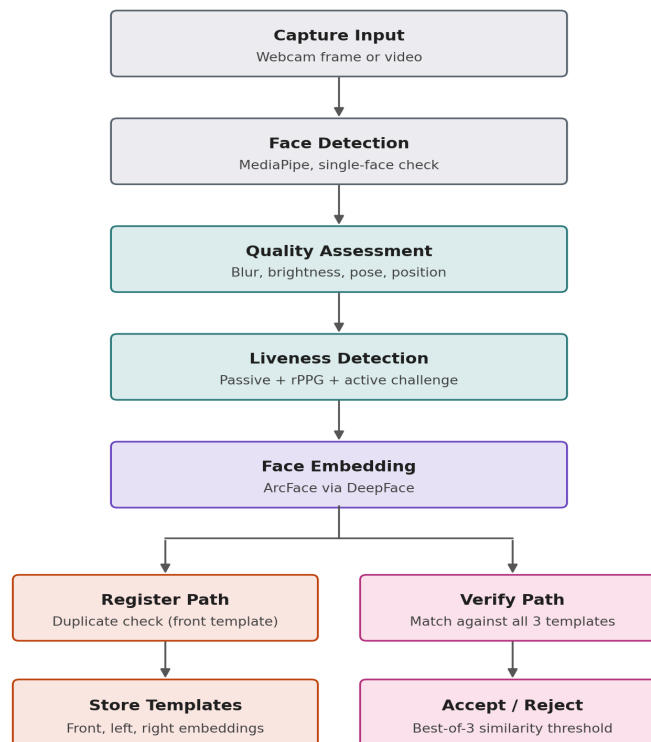
The reason this is hard is that a face-matching system alone can be tricked. Someone could hold up a printed photo, play a video, or display a photo on a phone screen, and a system that only checks 'does this face match' might be fooled. So the system also needs a way to catch these tricks, called liveness detection, and a way to make sure the photo it is working with is actually usable in the first place, called image quality assessment.

The project brief splits this into two phases, verification and registration. This design merges them into one shared pipeline with two entry points, since both phases reuse the same quality, liveness, and matching logic.

High-Level Architecture

Before any single feature is explained, it helps to see the whole shape of the system. There are two diagrams here: one shows the logical flow of data through the pipeline, and one shows how that pipeline is physically deployed as a running application on one machine.

Diagram 1 — Shared Core Pipeline



Both registration and verification share every stage up to Face Embedding, then split into two separate paths.

Capture Input: A single frame or short video from the webcam. This is the raw material everything else works on.

Face Detection: MediaPipe locates the face in the frame and immediately rejects frames with zero or more than one face, satisfying the brief's Single Face Validation requirement.

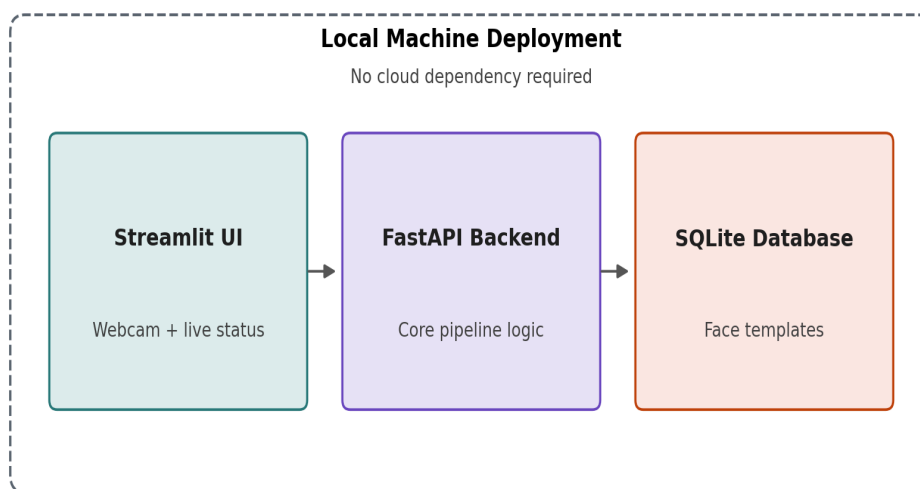
Quality Assessment: Brightness, blur, pose, position, and occlusion checks run here. A frame must pass this stage before liveness is even attempted, so time is not wasted running liveness checks on an unusable image.

Liveness Detection: Three layers run here: a pretrained passive model, a physiological rPPG check, and an active challenge (blink or head-turn). This satisfies section 4 of the brief, and is deliberately layered rather than relying on one method, for reasons explained fully in Section 5 of this document.

Face Embedding: Once a frame is confirmed to be good quality and live, DeepFace (using the ArcFace backend) converts the face into a numeric embedding, the representation used for matching.

The fork: This is the one deliberate structural change from the brief's two-phase description. Both paths share everything above; only the last step differs, because registering and verifying are, at their core, the same operation performed on a different side of a comparison (store a template versus compare against a template).

Diagram 2 — System Deployment Layers



Three components on one machine, satisfying the brief's explicit no-cloud-deployment requirement.

Streamlit UI: Handles the webcam feed and shows live, step-by-step status (quality check pass/fail, liveness pass/fail, match score) as a person registers or verifies. Chosen over React because the brief marks the UI as optional, and Streamlit reaches a working webcam-driven interface far faster. Open to using REACT as well.

FastAPI Backend: Exposes two endpoints, /register and /verify, and runs the actual pipeline from Diagram 1 underneath them.(at deployment stage can use Render/ Vercel free)

SQLite Database: Stores face templates locally, satisfying the brief's requirement that the system run without cloud deployment.(further can shift to supabase at the deployment stage free)

Multi-Angle Registration Design (Front, Left, Right)

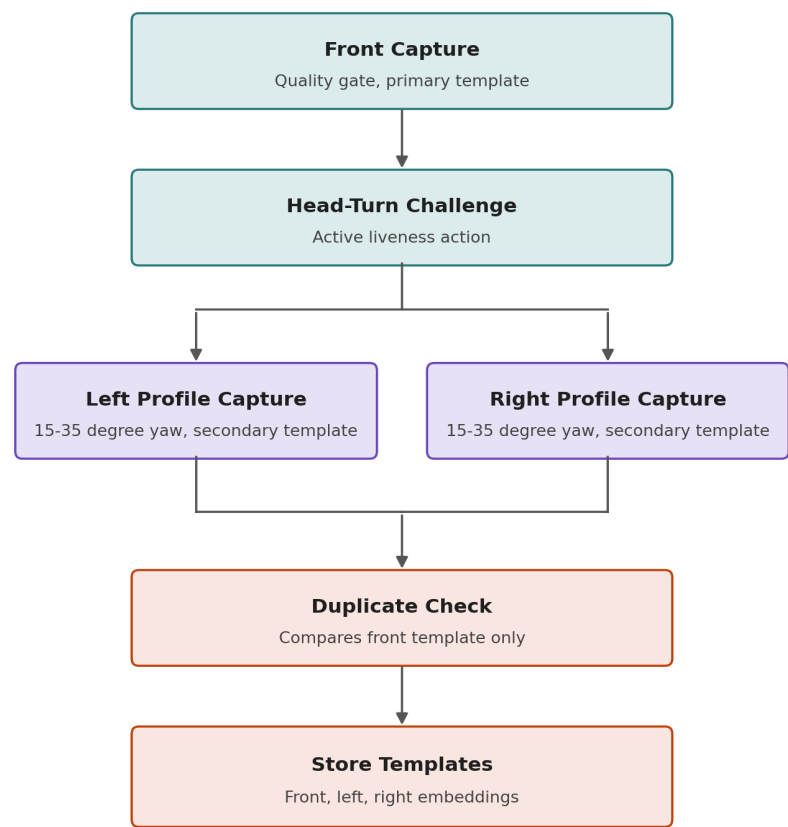
The Problem With a Single Enrollment Photo

If registration stores only one straight-on photo, verification only works reliably when the person is also facing the camera dead-on later. The moment someone verifies at even a slight angle, the match weakens, because a front-facing embedding is being compared against a fundamentally different-looking side-angle embedding. The brief requires the registration photo to be front-facing and explicitly rejects extreme profiles, which creates a real tension: robustness across angles is needed, but the quality gate cannot simply be relaxed to get it.

The Solution: Reuse the Active Liveness Challenge

The brief already requires an active liveness challenge where the user turns their head left and right. Instead of treating that purely as a liveness check and discarding the frames, the exact moment the head reaches a moderate turn angle is captured as a second embedding. One user action produces two outcomes: proof of liveness, and a genuinely useful side-angle enrollment template. No extra steps are added for the user.

Diagram 3 — The Full Registration Flow



Front capture stays strict (primary template); left and right captures are secondary templates from the existing head-turn challenge.

The Exact Angle Logic (solvePnP)

A weaker method would track the horizontal pixel position of the nose-tip landmark relative to the face edges and call a big shift a turn. This is a rough 2D approximation. The stronger, adopted method uses OpenCV's solvePnP function: it takes a set of known 3D reference points on a generic face model (nose tip, chin, eye corners, mouth corners) and matches them against the corresponding 2D MediaPipe landmark positions of the actual detected face, mathematically solving for real yaw, pitch, and roll angles in degrees.

This precision matters directly here: the left and right profile capture is triggered specifically when yaw is between 15 and 35 degrees. Below 15 degrees the angle is too close to frontal to be meaningfully different; above 35 degrees it approaches the extreme profile the brief rejects. This same 15 to 35 degree range is also what the CFP dataset comparison in Section 8 is benchmarked against.

Confirmed from direct repository study (Phase 2): MediaPipe's 468 landmarks have fixed, unchanging index numbers on every face. The four points solvePnP actually uses are landmark 1 (nose tip), landmark 152 (chin), landmark 33 (outer corner of the right eye), and landmark 263 (outer corner of the left eye). These four 2D points are matched against a generic 3D face model to solve for yaw, pitch, and roll. EAR (blink) calculation uses a separate set of six points around each eye. Knowing these exact index numbers, rather than treating MediaPipe as a black box, is what lets Chapter 8's occlusion check and Section 3.4's angle logic actually be implemented correctly.

Database Schema Change

Instead of one embedding per user, the schema stores three, linked to one identity:

Table	Columns	Purpose
users	user_id, name, created_at	One row per registered identity

templates	template_id, user_id (FK), angle_type (front/left/right), embedding, captured_at	Up to three rows per user, one per captured angle
-----------	--	---

Matching Logic Changes at Verification

At verification, the live embedding is compared against all three stored templates using cosine similarity, and the system accepts if the best of the three clears the threshold, rather than being compared against only a single frontal template. This is the actual payoff of the design: someone verifying at a slight angle is now compared against the closest-matching angle template.

Duplicate Detection Stays Simple

Duplicate checking only ever compares a new front template against existing front templates in the database. Checking all three angles per person would triple the comparison cost for no real benefit, since the front template alone is distinctive enough to catch an already-registered identity.

Image Quality Assessment

Every check below runs before liveness detection is attempted, so time is never spent analysing an image that is not even usable.

Check	What It Rejects (Brief Reference)	How It Works	Tooling	Limitation
Brightness	Reject frames that are too dark, overexposed, or backlit (brief 3.1)	Average pixel intensity of the image is calculated using OpenCV. Two thresholds are set: below one value means too dark, above another means overexposed.	OpenCV (cv2.mean)	Thresholds must be chosen from real measured values on self-collected good/bad images, not guessed; different cameras have different baseline brightness.
Blur	Reject motion-blurred or out-of-focus frames (brief 3.2)	Laplacian variance is computed on the grayscale image; a low value means edges are soft, i.e. the image is blurry.	OpenCV (cv2.Laplacian)	A single fixed threshold does not generalise perfectly across all cameras; needs calibration per device ideally.
Pose (yaw/pitch/roll)	Reject extreme profiles and excessive tilt, require front-facing (brief 3.3)	OpenCV solvePnP matches 3D reference face points to 2D MediaPipe landmark positions, solving for real yaw, pitch, and roll angles in degrees.	OpenCV solvePnP + MediaPipe landmarks	Needs a reasonably accurate generic 3D face model; extreme lighting can reduce landmark accuracy, indirectly affecting angle accuracy.
Position / Centering	Face must be centered and occupy sufficient frame area (brief 3.4)	The bounding box of the detected face is checked against the image center and against a minimum area threshold.	MediaPipe face detection	Simple geometric check; does not account for unusual camera framing conventions.
Single Face	Reject zero or multiple faces in frame (brief 3.5)	Counts the number of faces MediaPipe detects; exactly one is required to proceed.	MediaPipe face detection	A face partially visible at the frame edge could be missed or double-counted in rare cases.
Occlusion	Reject faces partially hidden by hands, hair, or masks (brief 3.6)	MediaPipe provides a visibility/confidence score per landmark; if key points around eyes, nose, or mouth fall below a threshold, the face is flagged as occluded.	MediaPipe landmark visibility scores	This is a scoped-down approximation, not a full trained occlusion classifier, due to time constraints; documented explicitly as a simplification.

Conceptual grounding: SDD-FIQA (CVPR 2021) argues quality should ultimately be tied to how recognizable a face is, not just how clean it looks. The checks above are a lightweight, classical-CV proxy for that deeper idea, chosen because a Deep Tiny Network study (arxiv.org/abs/2106.04852) showed even very small models can match large ones for quality scoring, reassuring that a lightweight, non-neural approach is a legitimate choice, not an underpowered shortcut.

Liveness Detection

Why One Method Is Never Enough

Seeing is Living? (arxiv.org/abs/2202.10673-Seeing is Living? Rethinking the Security of Facial Liveness Verification in the Deepfake Era

) demonstrates that real-time deepfake tools can fool liveness systems that only check for requested movement, since the fake action itself can be generated. This is the direct justification for combining three independent layers rather than trusting any single one.

Layer 1 — Passive Liveness (Pretrained Model)

A single captured image is passed to DeepFace with `anti_spoofing=True` (or the standalone MiniFASNet model), which returns a real-or-fake decision. Important technical note: DeepFace's anti-spoofing flag is not a separate technique from MiniFASNet, it runs MiniFASNet underneath, so these are effectively the same model accessed two different ways, not two independent alternatives.

Link: github.com/serengil/deepface and github.com/minivision-ai/Silent-Face-Anti-Spoofing

Limitation: Passive-only systems, as iProov's own research shows, can be fooled by real-time deepfake face-swapping combined with a virtual camera trick. This is exactly why it is only one of three layers here.

Naming clarification: The DeepFace Python library (by serengil) and Facebook's original 2014 DeepFace research paper (Taigman et al.) are unrelated beyond sharing a name; the library borrowed the name but is an independent, community-built codebase. This project uses the Python library, not Facebook's original model, and this distinction is worth stating explicitly to avoid implying otherwise.

Confirmed from direct repository study (Phase 2): MiniFASNet does not evaluate one image once; it crops the detected face at three different scales (1x, 2.7x, and 4x the face box) and runs a separate matching model variant on each crop, then fuses all three scores into one final decision. Its own documentation states it requires the face's rotation angle to stay within 30 degrees of vertical and the full face to be visible in frame, or results are not guaranteed. Separately, DeepFace's own recognition module raises a `hard ValueError` the moment a spoof is detected, rather than returning a gentle false result, so this project's own pipeline code must explicitly catch that exception and convert it into a normal reject-with-reason response instead of letting it crash the request.

Layer 2 — Physiological Liveness (rPPG)

Every heartbeat pushes a small pulse of extra blood through skin capillaries, causing a microscopic, invisible-to-the-eye change in skin colour in a rhythm matching the heartbeat. A printed photo or screen replay has no blood flow, so this signal does not exist in a spoofed image.

Exact pipeline: capture a still 10 to 15 second window, separate from the active head-turn challenge since motion corrupts this signal; extract a stable region of interest such as the forehead using existing MediaPipe landmarks; record the average green-channel pixel intensity per frame, building a time-series signal; apply a Butterworth bandpass filter (`scipy.signal`) restricted to the plausible human heart-rate range of roughly 0.7 to 4 Hz; run a Fast Fourier Transform (`numpy.fft`) and check for a clear, dominant peak in that band. A live face shows this peak; a photo shows a flat signal; a screen replay often shows no clean peak or an artifact tied to the screen refresh rate instead.

Links: arxiv.org/abs/2212.03680 (rPPG liveness survey section) and the broader review at [sciencedirect.com](https://www.sciencedirect.com)

Limitation: Webcam video compression can smear out the subtle colour signal; needs genuine stillness (any motion or lighting change corrupts it); should be treated as a supporting signal, not the primary gatekeeper.

Layer 3 — Active Liveness (Blink and Head-Turn)

Eye Aspect Ratio (EAR) is calculated from specific eye landmark points on every frame (Soukupová and Čech, 2016). A sharp, brief dip in this value across a short window of frames, not a single frame, indicates a blink. Head-turn detection uses the solvePnP-derived yaw angle described in Section 3.4. One challenge is picked at random and shown to the user as an instruction.

Links: Original EAR paper via [semanticscholar.org](https://www.semanticscholar.org); MediaPipe plus EAR confirmed as a proven combination in a 2024 replication (researchgate.net) and in SKINOPATHY AI (arxiv.org/abs/2603.00161), which confirms the specific 6-landmark EAR calculation.

Limitation: A single EAR threshold does not generalise perfectly across different eye shapes; a pre-recorded video of the correct action can, in principle, pass an active check alone, which is why it is combined with the other two layers.

Face Matching and Embeddings What Is an Embedding, in Plain Terms

An embedding is a list of numbers that represents a face's features. Two embeddings of the same person's face end up close together numerically; two embeddings of different people end up far apart. Comparing two embeddings with a similarity score is how matching actually works underneath.

Where This Idea Came From — FaceNet

FaceNet (Google, CVPR 2015) introduced this embedding concept to face recognition along with triplet loss, a training method that pulls same-identity embeddings closer and pushes different-identity embeddings apart during training. This project does not implement triplet loss directly, but understanding it explains why embeddings behave the way they do.

The Model Actually Used — ArcFace

ArcFace (CVPR 2019) improves on the FaceNet-era idea with Additive Angular Margin Loss, which adds an angular margin penalty during training for a clearer geometric separation between identities. This is the exact backbone DeepFace uses when ArcFace is selected as the model.

Link: arxiv.org/abs/1801.07698

Implementation: DeepFace with the ArcFace backend generates the embedding; the pretrained weights are used as-is, with no retraining or fine-tuning.

A Relevant Refinement — AdaFace

AdaFace adjusts the matching margin depending on image quality instead of treating every image equally. This is not implemented directly, but its core idea directly supports why this project runs quality checks before matching rather than after: quality should influence how strictly a face is judged, which is precisely what the quality-gate-then-match order in Diagram 1 already does.

Link: github.com/mk-minchul/AdaFace

Confirmed from direct repository study (Phase 2): AdaFace does not measure image quality with a separate model; it uses the norm (magnitude) of the face embedding vector itself, before normalization, as a stand-in for quality, tracked with a running average, and uses that to automatically loosen or tighten the matching margin per sample. Its own documentation also flags a practical, easy-to-miss gotcha directly relevant to this project: AdaFace expects images in BGR channel order, the same default OpenCV uses, whereas InsightFace's own models expect RGB, the default PIL uses. Mixing the two silently produces wrong results, which is a good habit to double-check whenever swapping between OpenCV-based and PIL-based preprocessing anywhere in this pipeline.

Similarity Scoring and Threshold Decisions

A live embedding is compared to stored template embeddings using cosine similarity. The accept or reject decision depends on a threshold, and that threshold is not guessed; it is calibrated using genuine and impostor pairs, explained fully in Section 11.

Registration and Duplicate Detection Workflow

Registration follows: capture front image, pass full quality gate, run the head-turn active liveness challenge while capturing left and right profile embeddings, run a duplicate check comparing the new front embedding against every existing stored front embedding using the same similarity function used at verification, and if no match exceeds the threshold, store all three embeddings under the new identity. This satisfies section 3 (Registration Validation Checks) and the optional Duplicate Face Detection requirement of the brief, treated here as a standard part of the flow rather than optional, since it is a low-cost, high-value addition once the matching function already exists.

Dataset Strategy — Four Sources, Four Distinct Jobs

A single dataset cannot answer every question this project needs answered: image quality, spoof detection, angle-based matching, and real device performance are four different questions. Instead of merging data into one pool, each source has exactly one job, and results are reported separately, never blended into one number.

Source	Link	Job	Trained or Data Only	How Used
CelebA-Spoof	github.com/ZhangYuanhan-AI/CelebA-Spoof	Passive liveness / spoof-detection evaluation	Data only, non-commercial research license	Balanced sample of ~300-500 images across lighting/spoof-type labels
CFP	cfpw.io / kaggle.com/datasets/chinafax/cfpw-dataset	Frontal-to-profile matching validation, directly proving the multi-angle design works	Data only	Frontal images as registered template, profile images as verification attempt
LFW	vis-www.cs.umass.edu/lfw	General genuine vs impostor matching baseline	Data only, free, no registration	~150-250 pairs for ROC curve and Equal Error Rate calculation
Self-Collected	Generated by the project	End-to-end pipeline testing, staged attacks, active liveness	Not a training dataset	Front/left/right captures across sessions, deliberately bad samples, staged attack samples

If recruiting a second real person is not possible, CFP's 500 identities double as an impostor pool against the one genuine self-collected identity, giving scaled impostor-rejection numbers that a single dataset alone could not provide.

Datasets Considered and Excluded, and Why

Choosing four sources was a deliberate decision made after comparing them against the classic datasets used throughout anti-spoofing research. None of the datasets below were used, and each was excluded for a specific, stated reason rather than simply not being known about.

Dataset	Scale	Reason Excluded
NUAA Photograph Imposter (2010)	15 subjects, ~12,600 images	Too small and narrow (photo attacks only); 15 people cannot represent real-world diversity
CASIA-FASD (2012)	50 subjects, 600 videos	Small subject pool and over a decade old; does not reflect modern phone cameras or attack quality
Replay-Attack / Idiap (2012)	50 subjects, 1300 videos	Same era limitation as CASIA-FASD: outdated camera quality and small scale
MSU-MFSD (2015)	35 subjects, 280 videos	Very small subject count; related MSU-USSA set also mixes genuine/attack capture devices, risking bias
SiW (2018)	165 subjects, ~4600 videos	Larger than the above, but RGB-only like the rest, and access requires a slower registration request
CASIA-SURF (2018)	1000 subjects, 21000 videos	Requires RGB + Depth + IR hardware; only the RGB portion would be usable with a standard webcam, wasting most of the dataset's value
HKBU-MARs V2	12 subjects, 1008 videos	Extremely small subject pool; narrow focus on 3D mask attacks only, a category explicitly out of scope
WMCA	72 subjects, 6716 videos	Needs thermal, depth, and near-infrared channels; completely impractical for a phone/webcam-only project

The consistent pattern across nearly all of these older datasets is a small subject pool and single visual modality, which is precisely the gap CelebA-Spoof (625,537 images, 10,177 subjects) was built to close. Where a genuinely useful capability existed (frontal-to-profile matching, in CFP's case) that the four chosen sources did not otherwise cover, that dataset was added rather than excluded, which is exactly how CFP earned its place in Section 8 above.

Attack Scenarios and Testing

Attack Type	Example	Expected Result	Layer That Should Catch It
Photograph	Printed photo, passport photo	Rejected	Passive liveness (texture cues)
Screen Replay	Photo displayed on phone or laptop	Rejected	Passive liveness, rPPG (no pulse)
Video Replay	Pre-recorded video played on a device	Rejected	rPPG, passive liveness
Frozen Frame	Paused video mid-playback	Rejected	Active liveness (no response to challenge)
Recorded Gesture	Pre-recorded blinking video	Rejected (known limitation)	rPPG as a secondary catch; active liveness alone may be fooled, honestly documented
Multiple Face	Two or more faces in frame	Rejected	Face detection stage
Poor Quality	Blurred, poorly lit, extreme angle	Rejected	Quality assessment stage

This project addresses physical presentation attacks (print, replay, video) only. Digital attacks such as deepfake injection are a distinct category needing different defenses, per the Unified Physical-Digital Attack Detection Challenge (arxiv.org/abs/2404.06211), and are explicitly out of scope.

Technology Stack

Component	Technology	Notes
Language	Python 3.11	3.10/3.11 used instead of 3.12 for MediaPipe/DeepFace compatibility
Computer Vision	OpenCV	Brightness, blur, solvePnP head pose
Face Detection & Landmarks	MediaPipe / Face Mesh	468-point landmarks, pretrained
Face Recognition & Matching	DeepFace + ArcFace	Pretrained embeddings and similarity scoring
Passive Liveness	DeepFace anti_spoofing (MiniFASNet underneath)	Pretrained spoof-detection model

Physiological Liveness	Custom rPPG (SciPy + NumPy)	Classical signal processing, not pretrained
Active Liveness	Custom (MediaPipe + EAR + solvePnP)	Built by us
Backend API	FastAPI + Uvicorn	/register and /verify endpoints
Database	SQLite	Local, no cloud dependency
User Interface	Streamlit + streamlit-webrtc	Used instead of React (optional in brief)
Evaluation	scikit-learn, matplotlib, numpy, pandas	ROC/EER, metrics tables
Dataset Access	kagglehub	CelebA-Spoof and CFP download

Deliberate Deviations From the Brief's Suggested Stack

- PyTorch (Model Development) — not used. No custom model is trained from scratch; pretrained DeepFace, ArcFace, and MiniFASNet are used instead, for realistic time-boxing within the available schedule.
- React (User Interface) — replaced with Streamlit, which the brief itself marks as optional, chosen for much faster development of a working webcam-driven interface.(open to change)

Evaluation Metrics, Fully Explained

APCER: Attack Presentation Classification Error Rate — how often an actual attack is wrongly accepted as genuine. Lower is better.

BPCER: Bona Fide Presentation Classification Error Rate — how often a genuine, real user is wrongly rejected. Lower is better.

ACER: Average Classification Error Rate — the average of APCER and BPCER, giving one balanced number.

ROC Curve / EER: Plots the tradeoff between false acceptance and false rejection at every possible threshold; the Equal Error Rate is the point where both are equal, and is used to select the final similarity threshold rather than guessing a number.

HTER: Half Total Error Rate — half the sum of false acceptance and false rejection rates, another standard cross-check metric.

AUC: Area Under the ROC Curve — a single number summarizing overall separability between genuine and impostor scores.

These terms come directly from the field's standard literature (Yu et al. survey, arxiv.org/abs/2106.14948, and Domain Generalization via Ensemble Stacking, arxiv.org/abs/2301.02145), used here instead of a single invented 'accuracy percent' figure.

Research Papers Referenced — Summary

Full breakdown (Problem, Method, Results, Limitations, What to Adopt, What to Avoid, Implementation Plan) for all 25 papers lives in the companion Research Paper Reference Table workbook. This section gives the condensed version: what each paper is, and exactly what was taken from it into this project.

Paper	Link	What Was Adopted Into This Project
CelebA-Spoof	arxiv.org/abs/2007.12342	Primary passive-liveness evaluation dataset (Section 8)
OULU-NPU	oulurepo.oulu.fi	Cited as the standard cross-condition generalization benchmark; not directly used
CVPR 2019 Multi-Modal FAS Benchmark	openaccess.thecvf.com	Confirms RGB-only scope matches mainstream research, justifying no depth/IR hardware
CLIPC8	arxiv.org/abs/2311.17583	Reference for classic dataset descriptions (NUAA, CASIA-FASD, Replay-Attack)
Yu et al. Survey (TPAMI)	arxiv.org/abs/2106.14948	Source of APCER/BPCER/ACER terminology adopted in Section 11
FPAD Survey 2025	pure.hud.ac.uk	Confirms RGB-camera liveness detection is a current, legitimate research area
MDPI Comprehensive Survey 2025	mdpi.com/2076-3417/15/12/6891	Basis for explicit scoping discipline in Section 9 and Section 13
Seeing is Living?	arxiv.org/abs/2202.10673	Direct justification for the 3-layer liveness design (Section 5.1)
Deep Frequent Spatial Temporal Learning	arxiv.org/abs/2002.03723	Source of the optional frequency-domain check stretch goal
Boulkenafet Colour Texture	github.com/KevinDapedri/...	Reference for an optional custom passive-liveness technique
CDCN	arxiv.org/abs/2003.04092	Cited as the state-of-the-art ceiling, justifying our simpler scoped approach
FPAD Survey, rPPG section	arxiv.org/abs/2212.03680	Core rPPG pipeline logic adopted directly (Section 5.3)
rPPG Review 2023	sciencedirect.com	Confirms classical, camera-only rPPG is legitimate, supporting our non-deep-learning approach
ArcFace	arxiv.org/abs/1801.07698	The actual matching backbone used throughout (Section 6.3)
GAC-FAS	arxiv.org/abs/2402.18817	Justifies testing our pipeline on our own self-collected data, not just published numbers
SDD-FIQA	ieeexplore.ieee.org	Conceptual framing for why quality checks matter (Section 4)
FIQA Literature Survey	dl.acm.org/10.1145/3575882.3575924	Authoritative citation backing the quality-check section
Deep Tiny Network FIQA	arxiv.org/abs/2106.04852	Justifies our lightweight, non-neural quality-check approach
Soukupova & Cech (EAR)	semanticscholar.org	The exact formula implemented for blink detection (Section 5.4)
Real-time Blink Detection 2024	researchgate.net	Confirms MediaPipe + EAR as a proven, current combination
SKINOPATHY AI	arxiv.org/abs/2603.00161	Confirms the specific 6-landmark EAR calculation
Unified Physical-Digital Attack Challenge	arxiv.org/abs/2404.06211	Basis for the physical-vs-digital attack scope statement (Section 9)
Benchmarking Unified Face Attack Detection	arxiv.org/abs/2505.13327	Confirms 3D mask attacks are a distinct, correctly excluded category
One-Class Knowledge Distillation	arxiv.org/abs/2205.03792	Conceptual reframing considered for active liveness design philosophy
Domain Generalization Ensemble Stacking	arxiv.org/abs/2301.02145	Source of the HTER and AUC metrics adopted in Section 11

Open-Source Projects and Tools Referenced — Summary

Full breakdown (Category, How Used, Why Important, Limitations) for all 21 tools and repositories lives in the companion Open-Source Tools & Repos sheet. Most of these are studied for context; only MediaPipe, OpenCV, and DeepFace are directly dependent on the running system.

Tool / Repo	Link	Usage Status	Role in This Project
InsightFace	github.com/deepinsight/insightface	Study only	Explains what ArcFace actually does under the hood, since DeepFace uses these same weights
MediaPipe	github.com/google-ai-edge/mediapipe	Direct use	Core dependency for every custom feature: quality, blink, head-turn, occlusion
OpenCV	github.com/opencv/opencv	Direct use	Blur detection, brightness, solvePnP head pose

RetinaFace	github.com/deepinsight/insightface/tree/master/detection/retinaface	Study only	Explains why landmark quality matters
SCRFD	github.com/deepinsight/insightface/tree/master/detection/scrfd	Study only	Modern comparison point against MediaPipe's detector
AdaFace	github.com/mk-minchul/AdaFace	Concept only	Directly informs the quality-gate-before-matching design philosophy
DeepfakeBench	github.com/SCLBD/DeepfakeBench	Future scope, not required	Relevant only if extended to digital/deepfake attacks
FaceX-Zoo	github.com/JDAI-CV/FaceX-Zoo	Study only	Context for why ArcFace was chosen over CosFace/SphereFace alternatives
PyImageSearch Examples	pyimagesearch.com	Direct reference	Hands-on code cross-check for EAR and head-pose implementation
dlib	github.com/davisking/dlib	Historical reference	Context for the original EAR paper's landmark approach
face_recognition	github.com/ageitgey/face_recognition	Historical reference	Simple intro to embedding-based matching before ArcFace
OpenVINO Face Demo	github.com/openvinotoolkit/open_model_zoo	Background only	Context on production inference optimization
Blink Detection (topic)	github.com/topics/blink-detection	Cross-check reference	Multiple independent EAR implementations to compare against
Head Pose Estimation (topic)	github.com/topics/head-pose-estimation	Direct reference	solvePnP implementation examples for yaw/pitch/roll
MediaPipe Face Mesh Examples	github.com/google-ai-edge/mediapipe/tree/master/examples	Direct reference	Correct landmark indexing for every custom feature
BRISQUE / OpenCV Quality Module	github.com/opencv/opencv_contrib/master/modules/quality	Optional, not used	Conceptual alternative to simple brightness/blur thresholding
FaceForensics++	github.com/ondyari/FaceForensics	Future scope, not required	Relevant only if extended to deepfake video detection

Three entries from the full reference sheet are omitted from the summary table above for brevity (PIQE, NIQE, and OpenFaceSwap), since none are used or have a well-maintained open-source implementation relevant to this project; they remain documented in full in the companion Open-Source Tools & Repos workbook sheet.

Phase 2 Findings Integrated Into This Design

Before this document moves into System Design, this section consolidates the concrete, code-level facts uncovered during Phase 2's repository study (Chapters 1-10 review of DeepFace, MediaPipe, OpenCV, MiniFASNet, InsightFace, and five reference projects) that directly changed or sharpened this design, rather than just adding background reading.

Finding	Where It Came From	How This Design Was Sharpened Because of It
Anti-spoofing raises a hard exception, not a graceful result	DeepFace modules/recognition.py	Pipeline code explicitly catches this exception and converts it into a normal reject-with-reason response, rather than assuming a clean is <code>real:false</code> return
Landmark indices 1, 152, 33, 263 are fixed and specific	MediaPipe Face Mesh, confirmed by direct index inspection	solvePnP head-pose code references these exact four points by number, not a vague nose/chin/eye description
Passive liveness fuses three crop scales (1x, 2.7x, 4x), not one	MiniFASNet src/generate_patches.py, src/anti_spoof_predict.py	Confirms passive liveness is already an ensemble internally; this project does not need to build its own multi-scale fusion on top
30 degree rotation limit stated by MiniFASNet's own authors	Silent-Face-Anti-Spoofing README	This becomes an explicit, testable precondition in the attack-testing matrix (Section 9), not an assumed constraint
AdaFace's quality-margin uses embedding feature norm, not a separate quality model	AdaFace head.py / inference.py	Confirms the quality-then-match ordering in Diagram 1 is conceptually aligned with published research, with a concrete mechanism to cite, not just a general idea
BGR vs RGB channel order differs between AdaFace/OpenCV and InsightFace/PIL	AdaFace repository documentation	Flagged as an explicit code-review checkpoint anywhere OpenCV-loaded and PIL-loaded images might mix in the same pipeline
SCRFD and RetinaFace both live inside InsightFace, not DeepFace	InsightFace detection/scrfd, detection/retinaface	Confirms these are background context only; MediaPipe remains the single detector used throughout this project for consistency

Full chapter-by-chapter detail (why each tool was built, folder structure, execution flow, weaknesses, what is borrowed versus built) for all ten repositories lives in the Phase 2 study notes; this table exists specifically to show how that reading translated into concrete design decisions, not reading for its own sake.

Commercial and Market Landscape

Before judging this project's own limitations, it helps to see where commercial, billion-dollar-backed products actually stand on this exact problem. None of them have fully solved it either, which sets a realistic bar for what this project should be expected to achieve.

Product	Link	Technology Used	Advantage	Disadvantage
iProov	iproov.com	"Flashmark" — flashes random colours of light on the face and reads how it bounces off real skin	Very hard to fool, works even against deepfakes, certified by independent labs	Needs a cloud connection, costs money, needs decent screen brightness/environment
FaceTec	facetec.com	Builds a 3D map of the face using a normal phone camera	Strong against printed photos and masks, since fake faces are flat	Has shown limitations against sophisticated real-time deepfake face swaps that maintain consistent 3D characteristics
Amazon Rekognition Face Liveness	AWS docs	Short video selfie following on-screen prompts, returns a 0-100 confidence score	Easy to integrate into any app already using AWS	Costs per check, needs internet/cloud; AWS itself states it cannot guarantee perfect results and should be combined with other checks
Azure Face API	Microsoft docs	Cloud-based face detection and matching, similar approach to AWS	Convenient if already using Microsoft cloud	Same cloud dependency and cost concerns as AWS
ID R&D / Innovatrics	via independent comparison articles	Passive facial liveness detection from a single image, no user action required	Fast, no prompts, good user experience	Passive-only systems have been shown to be foolable by real-time deepfake face-swapping combined with a virtual camera trick

The honest takeaway: even the best passive-only commercial systems claim false-acceptance rates only below roughly 0.05% under controlled lab conditions, and real attackers keep finding new tricks, such as the deepfake webcam-swap technique that has beaten passive-only products in independent testing. This project does not need to be perfect to be a legitimate proof of concept; it needs to

demonstrate a clear understanding of these exact tradeoffs, which is precisely why liveness here is built as three independent layers rather than one, and why every limitation in Section 15 below is stated openly rather than glossed over.

This also directly validates why the open-source building blocks chosen for this project (DeepFace's built-in anti-spoofing, MiniFASNet, and a custom active challenge) mirror the same layered philosophy used by serious commercial products, just at a smaller, local, non-commercial scale: nobody in this space, commercial or open-source, relies on a single method alone.

Limitations

Passive liveness relies on a pretrained model (DeepFace/MiniFASNet); it is not trained or validated by us from scratch, and its accuracy on our specific camera hardware is only confirmed through our own self-collected testing, not assumed from published numbers.

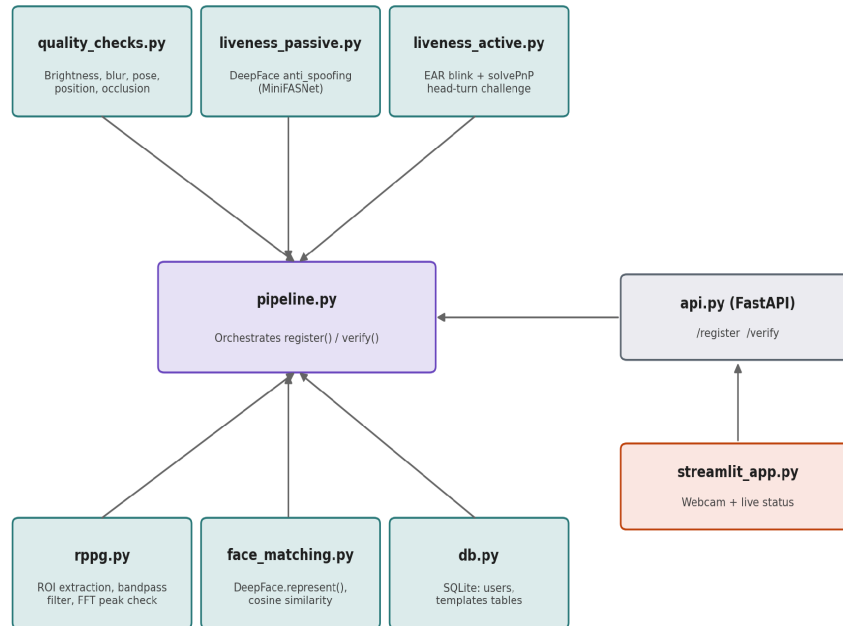
- Active liveness challenges (blink, head-turn) can, in principle, be defeated by a sufficiently advanced real-time deepfake or a pre-recorded video of the exact requested action, as demonstrated in published research (Seeing is Living?). This is mitigated, not eliminated, by combining active checks with passive and rPPG layers.
- rPPG liveness is sensitive to webcam compression, lighting stability, and motion; it is treated as a supporting signal, not a primary gatekeeper, and its real-world reliability on the specific hardware used is tested and reported honestly, not assumed.
- Occlusion detection is a landmark-visibility approximation, not a fully trained occlusion classifier, due to time constraints.
- Digital attacks (deepfake injection, AI-generated faces) and 3D mask attacks are explicitly out of scope; the system is only tested against physical presentation attacks (print, screen replay, video replay, frozen frame, multiple face).
- CelebA-Spoof is used under its non-commercial research and educational license, which is appropriate for this internship project but would need re-licensing for any commercial use.
- Self-collected data is necessarily small-scale (one to a few identities), so impostor-rejection and duplicate-detection numbers from self-collected data alone are indicative, not statistically robust; CFP's larger identity pool is used to partially offset this.

Future Scope

- Digital attack detection (deepfake and face-swap injection defenses), using DeepfakeBench and FaceForensics++ as a starting point, referenced in Section 13 but not implemented here.
- 3D mask attack detection, which would require specialized datasets (e.g. HKBU-MARs) and likely depth or infrared hardware not available in this project's scope.
- Training a custom, project-specific spoof-detection or quality-scoring model (using PyTorch, as originally suggested in the brief) once a larger, purpose-collected dataset exists, rather than relying solely on pretrained models.
- Multi-modal liveness detection (adding depth or infrared data, as in CASIA-SURF or WMCA) if suitable hardware becomes available.
- Cloud deployment and horizontal scaling, if the project moves beyond a local proof-of-concept toward a production multi-user system.
- Expanded self-collected data with multiple real participants, to make impostor-rejection and duplicate-detection metrics statistically stronger than what a single-identity or CFP-substituted test set can offer.

Module Diagram

This shows the codebase's internal file structure and which modules depend on which, distinct from Diagram 1's data flow and Diagram 4's request sequence.



pipeline.py is the single orchestrator; every quality, liveness, and matching module is independently testable and imported into it, never the reverse.

Folder Structure

```

face-verification-project/
├── data/
│   ├── celeba_spoof_sample/
│   ├── cfp_sample/
│   ├── lfw_sample/
│   └── self_collected/
├── src/
│   ├── quality_checks.py
│   ├── liveness_passive.py
│   ├── liveness_active.py
│   ├── rppg.py
│   ├── face_matching.py
│   ├── pipeline.py
│   └── db.py
├── api/
│   └── api.py
├── app/
│   └── streamlit_app.py
├── tests/
├── docs/
│   ├── Approach_Design_Document.docx
│   ├── Research_Paper_Reference_Table.xlsx
│   ├── Project_Plan.xlsx
│   └── requirements.txt

```